NirmanIQ Internship — Day 14 Handbook

MODULE	CORE TECHNOLOGY	AUTHOR
Security & Access Control	JWT, Passport, bcrypt & NestJS Guards	Penumuru Madhu Sudhan Reddy

Executive Summary: Day 14 secures our enterprise PostgreSQL backend. We implement salted bcrypt password hashing, stateless dual-token authentication (1h Access + 30d Refresh), request interception via NestJS CanActivate Guards, Role-Based Access Control (RBAC) across NirmanIQ's user hierarchy, refresh token rotation with replay detection, and custom @CurrentUser() parameter decorators.

1. The Dual-Token Authentication Pattern

Token Type	Lifespan	Storage & Transmission	Purpose & Mechanics
Access Token	1 Hour	`Authorization: Bearer <token>`	Stateless authorization. Verified cryptographically without database overhead. Lowers database strain.
Refresh Token	30 Days	httpOnly Cookie / Hashed in DB	Exchanges expired sessions for new token pairs at `POST /auth/refresh`. Uses token rotation.

2. NirmanIQ User Roles & Hierarchy (RBAC)

Role Code	Title	Permissions & Scope
`SE`	Site Engineer	Ground-level read access, daily progress logs. Cannot create or delete projects.
`PE`	Planning Engineer	First-tier review, schedule validation.
`PM`	Planning Manager	Second-tier approval: **Can CREATE projects** (`POST /projects`).
`PD`	Project Director	Multi-project oversight, cross-project analytics.
`CXO`	C-Suite Executive	High-level executive risk dashboards and portfolio views.
`TA`	Tenant Admin	Tenant organization configuration, user invite management.
`SA`	Super Admin	Full administrative authority: **Can DELETE projects** (`DELETE /projects/:id`).

3. Guard Implementation & Execution Flow

JwtAuthGuard (Authentication)

```
@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {
  handleRequest(err, user, info) {
    if (err || !user) {
      throw err || new UnauthorizedException(
        'Missing or invalid Bearer token'
      );
    }
    return user;
  }
}
```

RolesGuard (Authorization)

```
@Injectable()
export class RolesGuard implements CanActivate {
  canActivate(context: ExecutionContext): boolean {
    const roles = this.reflector.getAllAndOverride(ROLES_KEY, [
      context.getHandler(), context.getClass()
    ]);
    if (!roles) return true;
    const { user } = context.switchToHttp().getRequest();
    if (!user || !roles.includes(user.role)) {
      throw new ForbiddenException('Access denied');
    }
    return true;
  }
}
```

4. Clean Controllers with @CurrentUser()

Eliminate repetitive `req.user` extraction by leveraging custom parameter decorators:

```
@Controller('projects')
@UseGuards(JwtAuthGuard, RolesGuard)
export class ProjectsController {
  @Get()
  findAll(@CurrentUser() user: AuthenticatedUser) {
    return this.projectsService.findAll();
  }

  @Post()
  @Roles(Role.PM, Role.PD, Role.TA, Role.SA)
  create(@Body() dto: CreateProjectDto, @CurrentUser('userId') userId: string) {
    return this.projectsService.create(dto);
  }

  @Delete(':id')
  @Roles(Role.SA)
  delete(@Param('id') id: string) {
    return this.projectsService.delete(id);
```

```
    }
}
```

5. Refresh Token Rotation & Theft Prevention

Every time a refresh token is presented, the server validates its hash, issues a brand new access and refresh token pair, and overwrites the database hash. If an attacker attempts to replay an old refresh token, the server detects token theft and immediately revokes all active sessions for that user.